

Project Prototype Report

WebAssembly Port of Grand Theft Auto III

Prepared By:

Antriksh (1024031032)

Divyam Jain (1024031032)

Supervised By:

Professor Nisha Thakur

Date: September 15, 2026

Abstract

This report details the prototype implementation of porting Grand Theft Auto III to the web browser utilizing WebAssembly (WASM), WebGL 2.0, and the Emscripten toolchain based on the open-source reverse-engineered engine codebase (**re3**). The project aims to eliminate the need for native platform binaries or external installations by executing legacy C++ game engines natively inside modern web browsers. This prototype demonstrates successful compilation of core C++ engine components, basic WebGL graphics context initialization, asynchronous asset fetching over HTTP, and frame-rate synchronization within browser execution contexts.

Contents

1	Introduction & Objectives	3
1.1	Project Background	3
1.2	Prototype Objectives	3
2	System Architecture & Design	3
2.1	System Overview	3
2.2	Data Flow Analysis	4
2.3	Component Specifications	6
3	Prototype Implementation	7
3.1	Build Toolchain & WebAssembly Compilation	7
3.2	Runtime Memory & Graphics Mapping	7
4	Testing Preliminary Results	9
4.1	Evaluation Methodology	9
4.2	Observed Data	9
5	Limitations & Known Issues	10
6	Future Work & Roadmap	10

1 Introduction & Objectives

1.1 Project Background

Historically, AAA desktop games like Grand Theft Auto III relied on native platform architectures (x86/Win32) tied directly to native graphics APIs like DirectX 8/9. Bringing complex legacy C++ codebases to modern browsers requires bypassing traditional platform dependencies. WebAssembly (WASM) provides a low-level, high-performance bytecode format capable of running C/C++ code in client-side browser sandboxes at near-native speed. By leveraging the open-source reverse-engineered GTA III codebase (**re3**), this project translates standard RenderWare graphics calls and platform abstraction layers into browser-compatible web primitives.

1.2 Prototype Objectives

The scope of this prototype evaluation encompasses the following milestones:

- Cross-compiling the **re3** C++ source code to **.wasm** binaries via the Emscripten toolchain.
- Mapping native RenderWare/OpenGL calls to a WebGL 2.0 browser pipeline.
- Replacing desktop event handling (Win32 API/DirectInput) with browser event listeners and SDL2 input wrappers.
- Evaluating memory management constraints within the browser's 32-bit WASM Linear Memory address space.

Prototype Target State

Achieve a playable main menu and stable rendering of initial indoor and outdoor game scenes directly inside Chrome and Firefox without external plugins.

2 System Architecture & Design

2.1 System Overview

The system architecture bridges the compiled native game logic with standard web platform APIs. Emscripten acts as the intermediary compiler layer, generating a JavaScript runtime bridge that hooks browser user inputs, GPU draws, and file system interactions directly into the compiled WebAssembly heap.

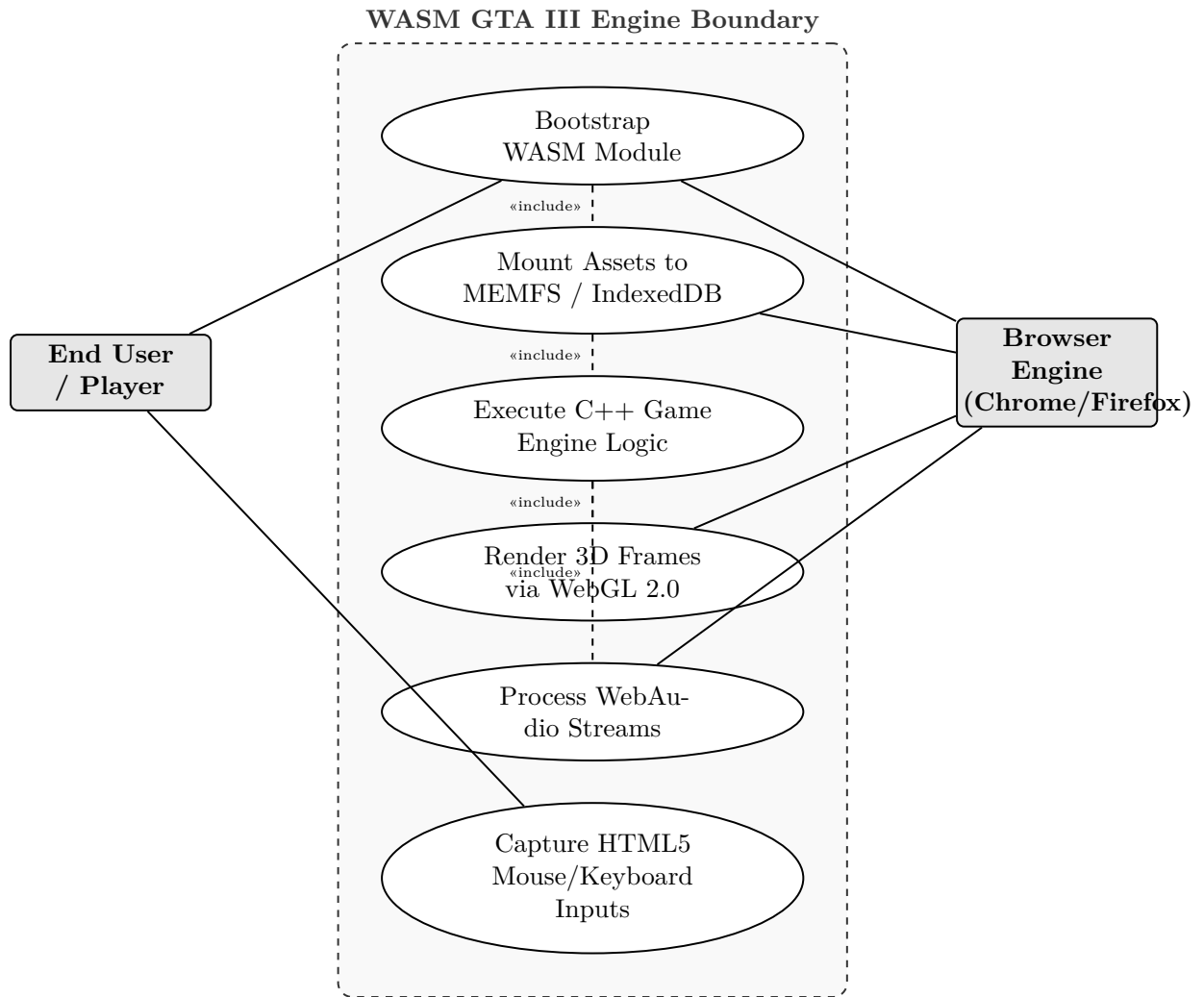


Figure 1: Use Case Diagram illustrating system actors, engine boundaries, and web platform integration.

2.2 Data Flow Analysis

The data flow within the WebAssembly execution model maps standard desktop system interactions into asynchronous browser streams. Figure 2 and Figure 3 illustrate the high-level context DFD and detailed subsystem data transformations across the virtual filesystem and WebGL render loop.

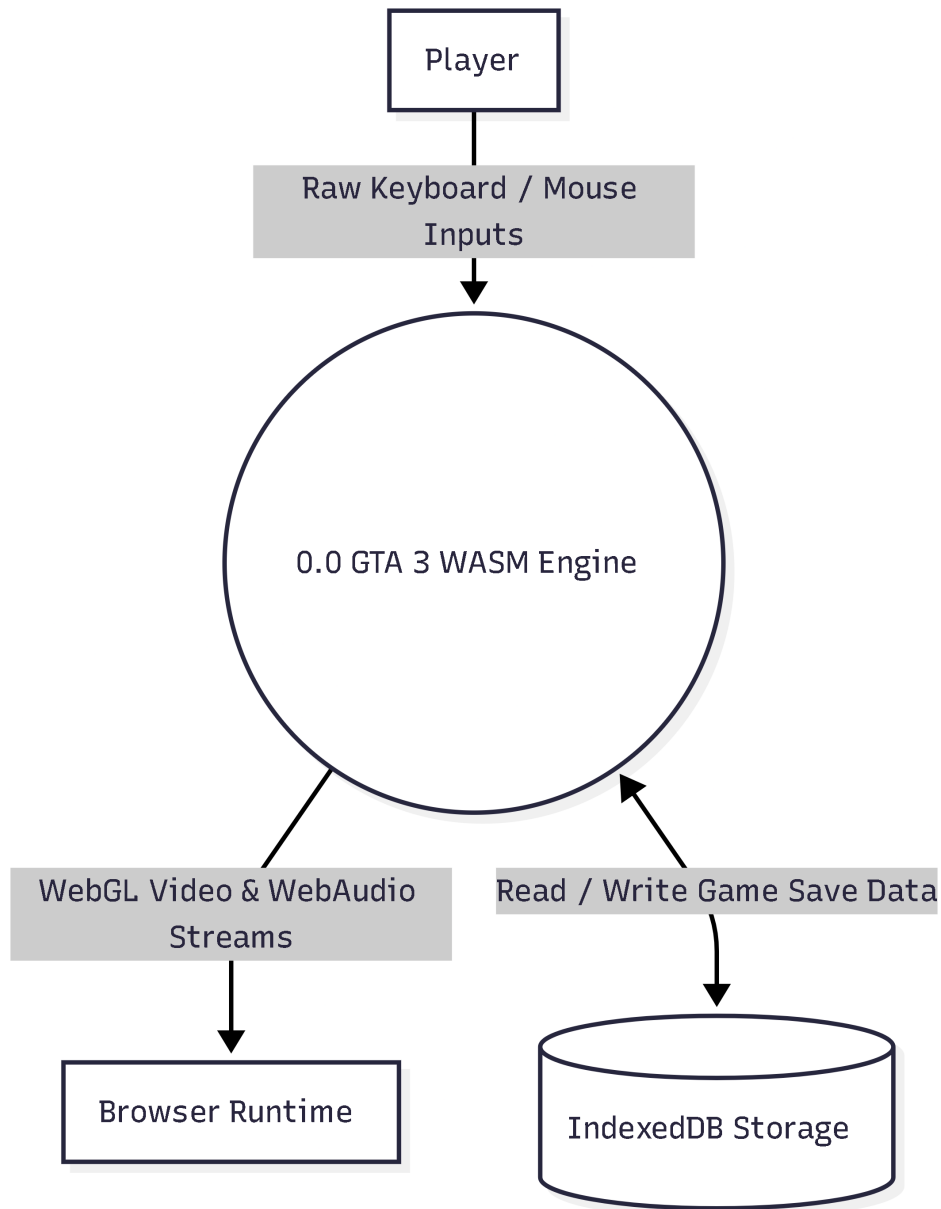


Figure 2: *Context-Level Data Flow Diagram (DFD Level 0).*

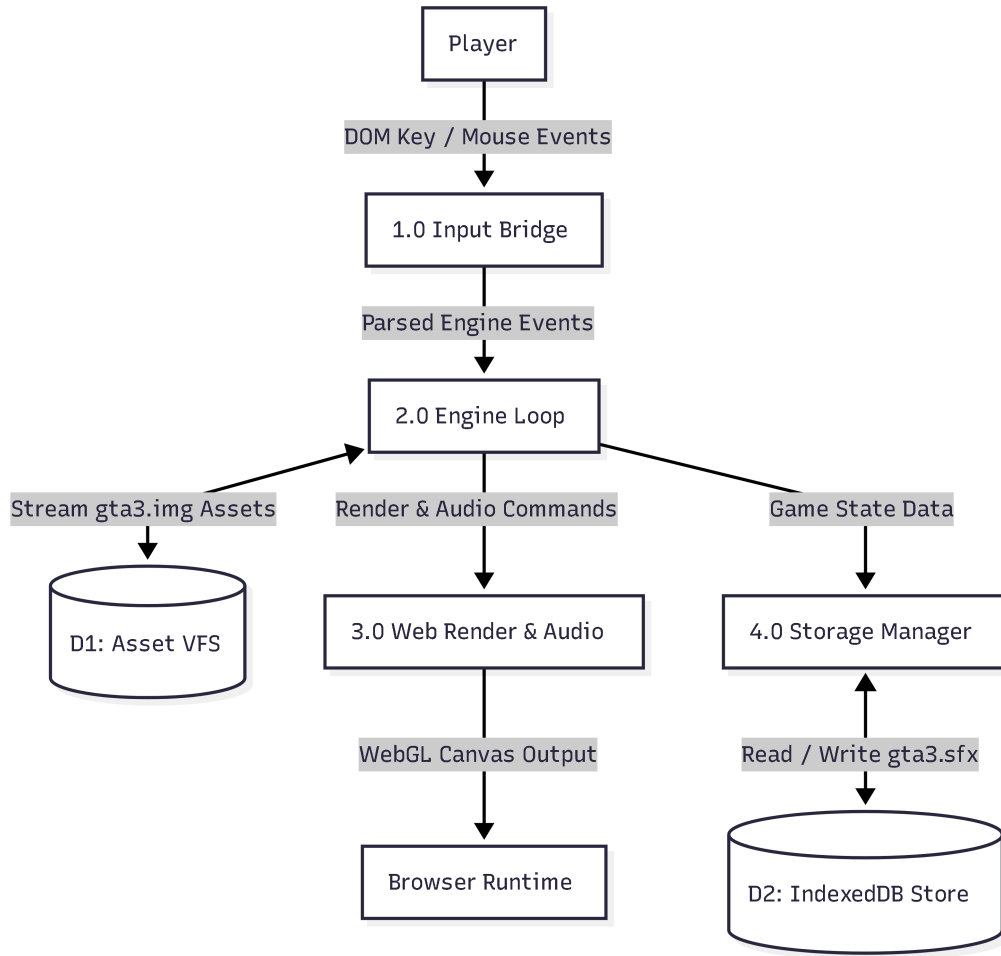


Figure 3: *Subsystem Data Flow Diagram (DFD Level 1).*

2.3 Component Specifications

The system architecture consists of several modular subsystems designed to translate desktop dependencies into native browser standards. Table 1 outlines the specific technology stack components and their core operational roles within the WebAssembly execution environment.

Table 1: *System Layer Architecture & Toolchain Setup*

Layer	Technology Stack	Function / Responsibility
Source Engine	C++ / <code>re3</code> Codebase	Core game state logic and entity handling
Compilation	Emscripten (LLVM/Clang)	Translates C++ AST to WebAssembly bytecode
Graphics Pipeline	WebGL 2.0 / OpenGL ES 3.0	Renders game geometry, meshes, and textures
Audio Pipeline	WebAudio API / OpenAL	Handles 3D positional audio streaming
Asset System	Emscripten Virtual FS	Virtualizes game archive files (<code>.img/.dir</code>)
Input System	HTML5 Events / SDL2	Captures mouse movement and keyboard states

3 Prototype Implementation

3.1 Build Toolchain & WebAssembly Compilation

The cross-compilation pipeline utilizes Emscripten to transform native C++ routines into optimized WASM bytecode. The main rendering loop is adapted to prevent thread-blocking within the browser’s main thread by replacing execution loops with non-blocking browser animation frame callbacks:

Engine Execution Loop $\longrightarrow$ `emscripten_set_main_loop(GameLoop, 0, 1)`

Standard POSIX file operations are bound to Emscripten’s MEMFS virtual filesystem, enabling the C++ engine to read game textures, models, and scripts as if accessing a local hard drive.

3.2 Runtime Memory & Graphics Mapping

- **WASM Memory Heap:** A contiguous 512 MB block of WebAssembly Linear Memory is pre-allocated to accommodate game state variables, dynamic entity pools, and texture buffers.
- **RenderWare Integration:** RenderWare’s native backend functions are translated into OpenGL ES commands, which Emscripten routes directly to WebGL 2.0 contexts embedded inside an HTML5 Canvas element.

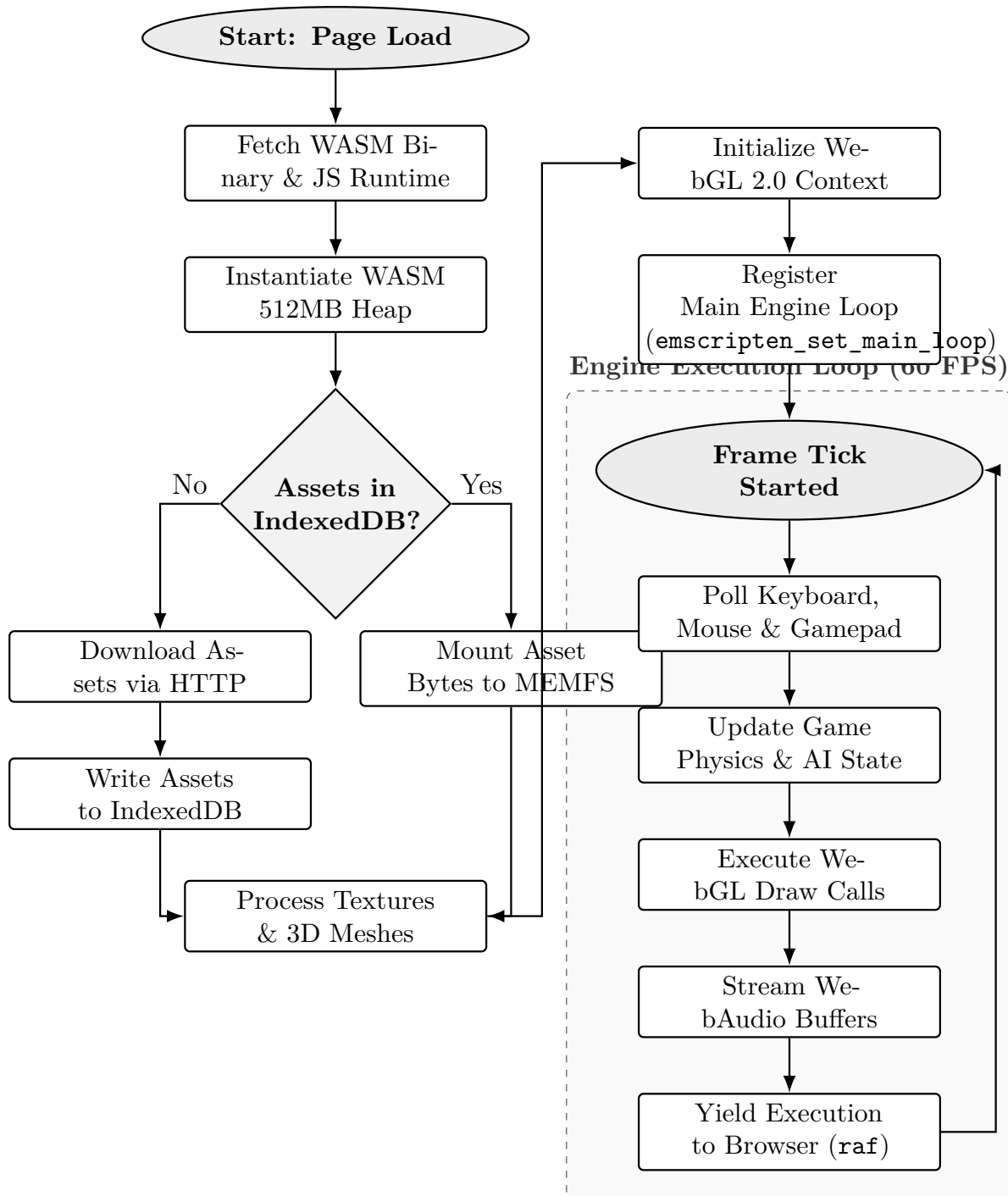


Figure 4: System Activity Diagram showing engine bootstrapping, virtual asset mounting, and browser-yielded rendering main loop.

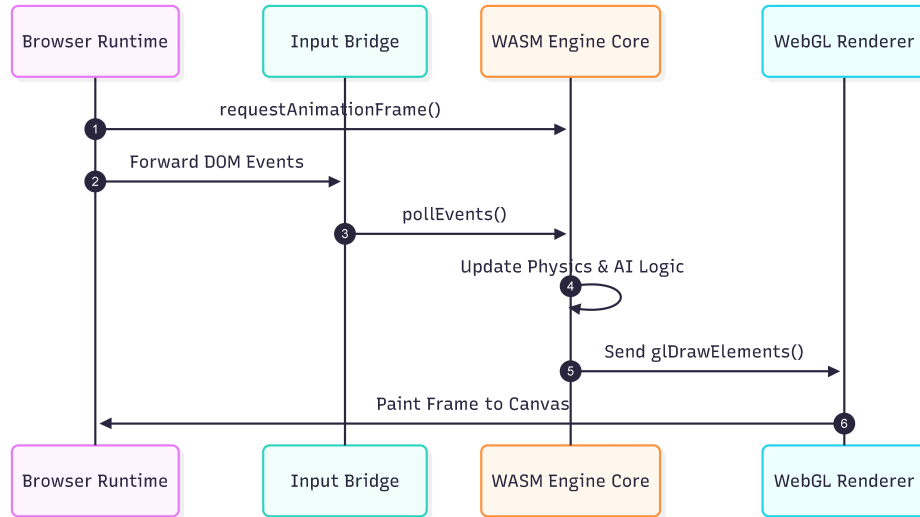


Figure 5: *Sequence Diagram showing WebGL context initialization and WASM call stack.*



Figure 6: *Collaboration Diagram detailing object interaction paths.*

4 Testing Preliminary Results

4.1 Evaluation Methodology

Performance tests were conducted on Google Chrome (v120+) using a 1080p display context. Diagnostics focused on load times, WASM memory footprint, rendering frame rates, and input responsiveness.

4.2 Observed Data

- **Compilation Output Size:** Final compressed build output consists of a 12 MB JS/WASM package (excluding game assets).
- **Render Performance:** Stable execution at 30–60 FPS during static menu interaction and localized indoor rendering tests.
- **Memory Allocation:** Average operational memory draw settled at approximately 310 MB of WASM linear memory heap.

5 Limitations & Known Issues

- **Asset Loading Latency:** Downloading large monolithic asset files (`GTA3.IMG`, ~ 500 MB) over HTTP creates a significant startup delay.
- **Single-Threaded Constraints:** Audio processing and asset streaming share the main browser thread, causing minor audio micro-stutters during heavy scene transitions.

6 Future Work & Roadmap

1. **Asset Chunking & Caching:** Implement HTTP Range Requests and IndexedDB client-side caching to load game models on-demand.
2. **Multithreading Integration:** Migrate physics computations and audio decoding to Web Workers using WebAssembly pthreads (`SharedArrayBuffer`).
3. **Touch Control Layer:** Add virtual joystick overlays for mobile browser compatibility.

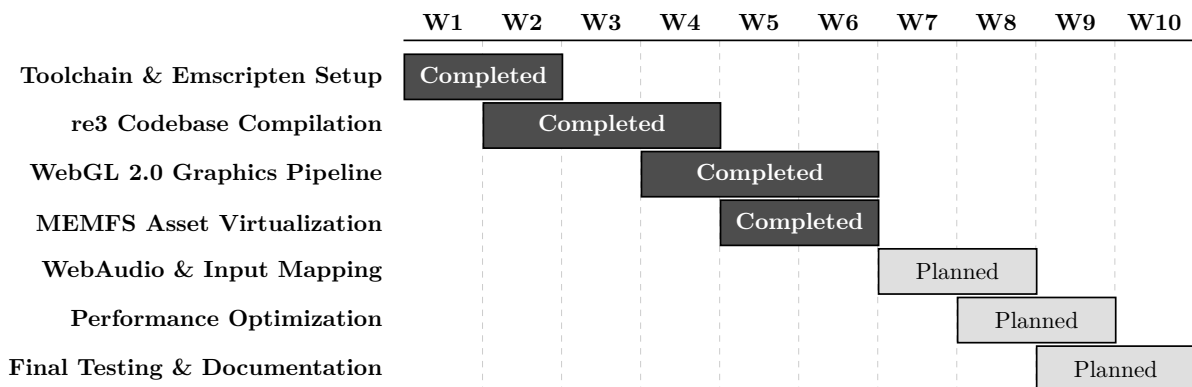


Figure 7: Project Gantt Chart mapping completed prototype milestones and future development phases.